

integration_sec09

9. Phase 8: Documentation, Deployment, and Operational Runbook

Phase 8 closes the integration lifecycle by producing every artifact required for sustainable operations. This phase encompasses constitution governance for all three repositories, submodule documentation propagation, user-facing guides, an operational runbook with metric-driven alerting, Docker and Kubernetes deployment manifests, a migration and rollback plan with phased rollout gates, and a 50-item master checklist. Every configuration option introduced across Phases 1–7 must appear in at least one documented reference table. Nothing ships without a signature.

9.1 Constitution and Governance Documentation

The constitution defines the behavioral contract for all code that interacts with the LLMsVerifier subsystem. It is not aspirational text; it is enforced through CI gates, challenge tests, and mandatory code-review checklists.

9.1.1 Write `internal/verifier/CONSTITUTION.md`

The file `internal/verifier/CONSTITUTION.md` contains 33 mandatory rules grouped into six categories, adapted from the HelixAgent reference constitution but contextualized for HelixTranslate's translation-domain requirements. Each rule carries a unique identifier, category, imperative text, rationale, enforcement mechanism, and violation consequence. The eight scoring rules (S01–S08) govern weight validation, threshold enforcement, score freshness with TTL-based recency penalties, normalization boundaries, component isolation to prevent cross-contamination of speed and cost signals, recalculation triggers on discovery events, and audit-trail persistence for every computed score. The eight verification rules (V01–V08) mandate pipeline integrity across all eight steps, enforce step isolation so a failure in latency benchmarking cannot mask an authentication failure, specify timeout values per step (5s for existence, 120s for latency), limit retries to three attempts per critical step, prohibit mock-only verification (anti-bluff), require a minimum of 193 passing challenges, enforce challenge coverage across all five scoring components, and mandate persistence of every verification result to SQLite. The five discovery rules (D01–D05) regulate refresh intervals per provider, establish fallback ordering from static registry to provider API to `models.dev`, require deduplication by model ID, define stale handling when a model has not been

rediscovered within $2\times$ the provider's refresh interval, and mandate provider health tracking with automatic degradation of unhealthy sources. The five selection rules (L01–L05) enforce fairness in the selection algorithm, require a diversity minimum of three distinct providers in any ranked list, implement load balancing across top-scored models, express tier preference for premium-tier models on quality-critical translations, and preserve user override capability. The four testing rules (T01–T04) codify the anti-bluff mandate, require 100% test coverage for the internal/verifier/ package, mandate real infrastructure for all integration tests, and prohibit mock-only verification pipelines. The three operations rules (O01–O03) require health endpoint monitoring, define alerting thresholds for score drops exceeding 10% within one hour, and mandate rollback capability through the LLMSVERIFIER_ENABLED feature flag.

9.1.2 Constitution Rule Template

Every rule in internal/verifier/CONSTITUTION.md follows the template shown in Table 9.1. This structure ensures that each rule is actionable, auditable, and enforceable without ambiguity.

Table 9.1: Constitution Rule Template with Representative Entries

ID	Category	Rule Text	Rationale	Enforcement Mechanism	Violation Consequence
S01	Scoring	All five component weights must sum to exactly 1.0	Normalization integrity prevents skewed composite scores	validateWeights() in internal/verifier/scoring/components.go returns error on mismatch	CI build fails; blocked
S04	Scoring	Recency penalty applies exponential decay with half-life 90 days	Older models depreciate naturally without manual intervention	CalculateRecencyScore() enforces time-decay formula; scores older than 24h are recalculated	Stale scores trigger warning alert; model may fall below threshold
V01	Verification	All eight pipeline steps must execute for every model before registry admission	Incomplete verification creates blind spots in capability detection	RunPipeline() in internal/verifier/pipeline.go requires all steps complete	Model cannot enter verified_model table; gatekeeper denies access
V06	Verification	A minimum of 193 challenges must pass before a model	Challenge count below 193 produces statistically unreliable capability metrics	internal/verifier/challenges/runner.go counts passes; returns error if count < 193	Score component set to 0.0; model flagged as unverified

ID	Category	Rule Text	Rationale	Enforcement Mechanism	Violation Consequence
T01	Testing	receives a capability score No test may rely exclusively on mocked LLM responses for verification pipeline coverage	Mock-only tests produce false confidence; anti-bluff mandate requires real behavior	go test -cover + manual review; integration tests require live verifier instance	PR rejected; coverage report flagged
O03	Operations	Rollback to legacy provider factory must complete within 30 seconds via feature flag	Sustained degradation of verified model quality demands instant fallback	LLMSVERIFIER_ENABLED=false takes effect in next provider factory call; no restart required	Operational incident if rollback exceeds 30s SLA

9.1.3 Update CLAUDE.md

The root CLAUDE.md file (11KB) receives a new appendix section titled "LLMsVerifier Integration Context." This section documents the architecture context: the verifier subsystem lives under internal/verifier/ and provides a verified, scored, discovered model layer between pkg/translator/llm/ and external LLM APIs. Key files are listed with one-line descriptions for all 15+ integration files: internal/verifier/client.go (verifier client), internal/verifier/pipeline.go (8-step pipeline), internal/verifier/scoring/engine.go (scoring engine), internal/verifier/scoring/components.go (5 component calculators), internal/verifier/scoring/composite.go (weighted aggregator), internal/verifier/discovery/service.go (3-tier discovery), internal/verifier/discovery/registry.go (model registry), internal/verifier/discovery/gatekeeper.go (access control), internal/verifier/selection/engine.go (model selector), internal/verifier/selection/fallback.go (fallback chain), internal/services/llmsverifier_score_adapter.go (score adapter), internal/verifier/config.go (verifier config), internal/verifier/health.go (health checks), internal/verifier/events.go (event handlers), and internal/verifier/CONSTITUTION.md (governance rules). The pipeline description covers the eight ordered steps (existence → connectivity → authentication → completion → capability detection → translation quality → latency benchmark → error handling) with per-step timeouts and criticality flags. The scoring weight reference table documents the default distribution: response speed 0.25, cost effectiveness 0.25, model efficiency 0.20, capability 0.20, recency 0.10. Common troubleshooting steps cover five scenarios: verifier client connection refused (check LLMSVERIFIER_API_URL), model not appearing in registry (run discovery or check

gatekeeper threshold), score lower than expected (review component breakdown in logs), pipeline timeout on latency step (increase LLMsverifier_TIMEOUT), and SQLite lock contention (reduce MaxConcurrentTests).

9.1.4 Update AGENTS.md

The root AGENTS.md file (20KB) receives a new appendix section titled "Verifier Agent Roles and Decision Trees." This section defines three agent roles. The Discovery Agent is responsible for monitoring the 3-tier discovery system, reviewing discovery logs in internal/verifier/discovery/, and triggering manual discovery via the API when automated refresh fails. Its decision tree: on ModelDiscoveryFailed event, check provider API key validity → if valid, check provider health endpoint → if healthy, retry discovery → if still failing, log incident and proceed with cached models. The Verification Agent executes the 8-step pipeline, reviews step results, and investigates failures. Its decision tree: on VerificationStepFailed event, identify failed step → if critical step, block model and alert → if non-critical, log warning and continue → always publish results. The Scoring Agent monitors score trends, investigates drops, and recommends weight adjustments. Its decision tree: on ScoreBelowThreshold event, check recency (is score stale?) → if stale, trigger recalculation → if fresh, check component breakdown → identify declining component → recommend action (retire model, adjust weights, or escalate). Context window constraints are specified: each agent receives at most 4,000 tokens of log context and must reference specific line numbers in source files when making recommendations.

9.2 Submodule Documentation Propagation

The HelixTranslate project maintains two Git submodules: Challenges/ (at commit 3937f06) and Containers/ (at commit f572d26). Both submodules require their own CONSTITUTION.md, CLAUDE.md, and AGENTS.md updates. The LLMsVerifier repository itself (checked out at ./LLMsVerifier/) is treated as a third submodule for documentation purposes.

9.2.1 Submodule Documentation Checklist

Table 9.2 enumerates the documentation requirements for each submodule. Every cell must carry a commit hash and timestamp at completion.

Table 9.2: Submodule Documentation Completeness Matrix

Document	Challenges Submodule	Containers Submodule	LLMsVerifier Submodule	Completion Gate
CONSTITUTION.md	Must add test integrity rules (T01–T04), anti-bluff mandates, and challenge authoring guidelines	Must add container build rules, base image pinning rules, and security scanning rules	Must update existing constitution with HelixTranslate integration rules (S01–	Signed commit on main branch

Document	Challenges Submodule	Containers Submodule	LLMsVerifier Submodule	Completion Gate
CLAUDE.md	specific to translation evaluation		S08, V01–V08)	
	Must append challenge system context: how to add challenges, how to run the challenge runner, expected pass criteria format	Must append container build context: multi-stage build rules, dependency cache optimization, base image selection	Must append integration context: how HelixTranslate consumes LLMsVerifier APIs, key exported functions, module boundary rules	PR merged; code review passed
	Must define Challenge Author Agent role with decision tree for authoring new translation-quality challenges	Must define Build Agent role for container build and release decisions	Must define Provider Integration Agent role for adding new provider backends	AI agent smoke test passed
Submodule references in main project	.gitmodules entry verified	.gitmodules entry verified	go.mod replace directive verified	git submodule status returns clean

9.2.2 Write Challenges Submodule CONSTITUTION.md

The Challenges submodule CONSTITUTION.md focuses on test integrity. Rule TC01 mandates that every challenge must define an unambiguous pass criterion expressed as a deterministic check against expected output, never as a subjective quality assessment. Rule TC02 prohibits challenges that rely on external network state; each challenge must declare its infrastructure dependencies in a REQUIREMENTS block. Rule TC03 requires that adding, removing, or modifying any challenge triggers a full re-verification of all 193+ challenges against a reference model to detect regression in challenge validity. Rule TC04 mandates anti-bluff design: no challenge may pass against a hardcoded response or a model that is known to be non-functional.

9.2.3 Write LLMsVerifier Submodule Documentation Updates

The LLMsVerifier repository receives two documentation updates. In CLAUDE.md, a new section titled "HelixTranslate Integration Context" documents the module boundary: digital.vasic.llmsverifier exports a public API surface consumed

by `digital.vasic.translator` through the adapter in `internal/providers/llmsverifier/adapter.go`. Key exported types are listed: `llmverifier.Client`, `llmverifier.CompletionRequest`, `llmverifier.CompletionResponse`, `scoring.ScoringEngine`, `scoring.ComprehensiveScore`, `discovery.Service`, and `verification.Verifier`. In `AGENTS.md`, a `Provider Integration Agent` role is defined with the decision tree: on request to add provider N+1, check if provider implements OpenAI-compatible API → if yes, create adapter using existing OpenAI client pattern → if no, implement native provider in `providers/` directory → run 8-step verification → add to static registry → update documentation.

9.2.4 Documentation Update Checklist Script

A validation script ensures all submodule documentation files remain in sync with the main project. The script checks file existence, minimum line counts, reference cross-links, and timestamp freshness. Code block 9.1 shows the complete validation script.

```
#!/usr/bin/env bash
# scripts/validate-documentation.sh
# Validates that all CONSTITUTION.md, CLAUDE.md, and AGENTS.md
#   files
# are present and cross-referenced across main project and
#   submodules.
```

```
set -euo pipefail
```

```
REQUIRED_FILES=(
  "internal/verifier/CONSTITUTION.md"
  "CLAUDE.md"
  "AGENTS.md"
  "Challenges/CONSTITUTION.md"
  "Challenges/CLAUDE.md"
  "Challenges/AGENTS.md"
  "Containers/CLAUDE.md"
  "Containers/AGENTS.md"
  "LLMsVerifier/CLAUDE.md"
  "LLMsVerifier/AGENTS.md"
)
```

```
MIN_LINES=(150 200 300 100 150 200 80 120 200 200)
```

```
ERRORS=0
```

```
for i in "${!REQUIRED_FILES[@]}; do
  FILE="${REQUIRED_FILES[$i]}"
  MIN="${MIN_LINES[$i]}"
  if [[ ! -f "$FILE" ]]; then
    echo "ERROR: Missing required file: $FILE"
    ERRORS=$((ERRORS + 1))
  elif [[ "$(wc -l < "$FILE")" -lt "$MIN" ]]; then
    echo "ERROR: $FILE has fewer than $MIN lines"
    ERRORS=$((ERRORS + 1))
  else
```

```

    echo "OK: $FILE ($(wc -l < "$FILE") lines)"
  fi
done

# Verify cross-references
if ! grep -q "LLMsVerifier" CLAUDE.md; then
  echo "ERROR: CLAUDE.md missing LLMsVerifier section"
  ERRORS=$((ERRORS + 1))
fi

if ! grep -q "internal/verifier/CONSTITUTION.md" AGENTS.md; then
  echo "ERROR: AGENTS.md missing verifier constitution reference"
  ERRORS=$((ERRORS + 1))
fi

if [[ $ERRORS -gt 0 ]]; then
  echo "Documentation validation FAILED with $ERRORS error(s)"
  exit 1
fi

echo "Documentation validation PASSED"

```

9.3 User-Facing Documentation

User-facing documentation explains how the LLMsVerifier integration affects translation workflows, what verified models mean, how to interpret scores, and how to diagnose common issues.

9.3.1 Write docs/llmsverifier-integration.md

The file docs/llmsverifier-integration.md provides a high-level overview for end users. It explains that verified models are those that have passed all eight steps of the verification pipeline: existence confirmation, API connectivity, authentication validation, basic completion, capability detection, translation quality assessment, latency benchmarking, and error handling verification. It describes how each model receives a composite score from 0.0 to 10.0, computed from five weighted components (response speed 25%, cost effectiveness 25%, model efficiency 20%, capability 20%, recency 10%). It documents the model selection UX: when a user initiates a translation, only verified models with scores above their tier threshold appear in the provider selection list. Unverified models are hidden by default and require explicit `show_unverified=true` query parameter to appear, at which point they display a warning badge. Score badges use a color scheme: excellent (9.0–10.0, green), very good (7.5–8.9, blue), good (6.0–7.4, yellow), fair (4.0–5.9, orange), poor (below 4.0, red). The filtering system allows users to filter by provider, minimum score, capability tag (e.g., long-context, json-mode, streaming), and cost tier.

9.3.2 Environment Variables Reference

Every environment variable introduced or consumed by the LLMsVerifier integration appears in Table 9.3. This table is the single source of truth for operational configuration.

Table 9.3: Complete Environment Variables Reference (33+ Variables)

Variable Name	Type	Default	Required	Description
LLMSVERIFIER_ENABLED	bool	false	No	Master enable switch for the LLMsVerifier subsystem
LLMSVERIFIER_API_URL	string	http://localhost:8080	Yes	Base URL of the LLMsVerifier service
LLMSVERIFIER_API_KEY	string	""	Yes	API key for authentication with LLMsVerifier
LLMSVERIFIER_DB_PATH	string	./data/verifier.db	No	SQLite database path for local score caching
LLMSVERIFIER_CACHE_TTL	duration	1h	No	Time-to-live for cached scores before recalculation
LLMSVERIFIER_VERIFICATION_ENABLED	bool	true	No	Enable the 8-step verification pipeline
LLMSVERIFIER_SCORING_ENABLED	bool	true	No	Enable 5-component score calculation
LLMSVERIFIER_DISCOVERY_ENABLED	bool	true	No	Enable 3-tie model discovery
LLMSVERIFIER_MAX_CONCURRENT	int	5	No	Maximum concurrent verification tests
LLMSVERIFIER_TIMEOUT	duration	30s	No	Per-step verification timeout
LLMSVERIFIER_WEIGHT_SPEED	float	0.25	No	Scoring weight:

Variable Name	Type	Default	Required	Description
				response speed
LLMSVERIFIER_WEIGHT_COST	float	0.25	No	Scoring weight: cost effectiveness
LLMSVERIFIER_WEIGHT EFFICIENCY	float	0.20	No	Scoring weight: model efficiency
LLMSVERIFIER_WEIGHT_CAPABILITY	float	0.20	No	Scoring weight: capability
LLMSVERIFIER_WEIGHT_RECENCY	float	0.10	No	Scoring weight: recency
OPENAI_API_KEY	string	""	Yes (if using OpenAI)	OpenAI API authentication key
ANTHROPIC_API_KEY	string	""	Yes (if using Anthropic)	Anthropic API authentication key
DEEPSEEK_API_KEY	string	""	Yes (if using DeepSeek)	DeepSeek API authentication key
GROQ_API_KEY	string	""	Yes (if using Groq)	Groq API authentication key
MISTRAL_API_KEY	string	""	Yes (if using Mistral)	Mistral API authentication key
COHERE_API_KEY	string	""	Yes (if using Cohere)	Cohere API authentication key
XAI_API_KEY	string	""	Yes (if using xAI)	xAI API authentication key
TOGETHER_API_KEY	string	""	Yes (if using Together)	Together AI API key
OPENROUTER_API_KEY	string	""	Yes (if using OpenRouter)	OpenRouter API key
CLOUDFLARE_API_KEY	string	""	Yes (if using Cloudflare)	Cloudflare API key
SAMBA NOVA_API_KEY	string	""	Yes (if using SambaNova)	SambaNova API key
NOVITA_API_KEY	string	""		

Variable Name	Type	Default	Required	Description
			Yes (if using Novita)	Novita API key
MOONSHOT_API_KEY	string	""	Yes (if using Moonshot)	Moonshot API key
QWEN_API_KEY	string	""	Yes (if using Qwen)	Qwen API authentication key
REPLICATE_API_TOKEN	string	""	Yes (if using Replicate)	Replicate API token
HYPERBOLIC_API_KEY	string	""	Yes (if using Hyperbolic)	Hyperbolic API key
CEREBRAS_API_KEY	string	""	Yes (if using Cerebras)	Cerebras API key
SILICONFLOW_API_KEY	string	""	Yes (if using SiliconFlow)	SiliconFlow API key
FIREWORKS_API_KEY	string	""	Yes (if using Fireworks)	Fireworks API key
PERPLEXITY_API_KEY	string	""	Yes (if using Perplexity)	Perplexity API key
AI21_API_KEY	string	""	Yes (if using AI21)	AI21 Studio API key
ZHIPU_API_KEY	string	""	Yes (if using Zhipu)	Zhipu API authentication key

9.3.3 Document Model Selection UX

The model selection user experience follows a gatekeeping pattern. When a user opens the provider selection interface, the system queries `internal/verifier/discovery/registry.go` for all models where `verification_status = 'verified'` and `composite_score >= tier_threshold`. Models failing either condition are excluded from the default list. Each displayed model shows its name, provider, composite score badge (color-coded per range from Section 9.3.1), capability tags (chips for streaming, json-mode, long-context, function-calling), and cost tier indicator (premium, standard, budget). Users may apply filters: provider multi-select, minimum score slider (0–10), capability tag toggle, and cost tier multi-select. The sort default is composite score descending; alternatives are latency ascending, cost per token ascending, and recency descending. Users may override the gatekeeper by enabling "Show All Models" in advanced settings, which reveals unverified models with a red warning badge and requires explicit confirmation before use.

9.3.4 Write docs/troubleshooting.md

The troubleshooting guide covers ten common issues. Issue 1: "Model does not appear in selection list" — diagnosis: check verification status via `GET /api/v1/models/{id}`, check composite score against tier threshold, check discovery logs; resolution: trigger manual discovery or lower the minimum score filter. Issue 2:

"Model score dropped unexpectedly" — diagnosis: check `internal/verifier/scoring/history.go` for trend data, review recency penalty application, check if provider pricing changed; resolution: force score recalculation via `POST /api/v1/score/calculate` or temporarily reduce weight on the declining component. Issue 3: "Discovery service returns no models for a provider" — diagnosis: check provider API key validity, check provider health endpoint response, review `discovery_service.go` logs for timeout or HTTP error; resolution: renew API key, increase provider timeout, or fall back to static registry. Issue 4: "API key reported as invalid" — diagnosis: verify key format matches provider expectations, check key has not expired, confirm key has required scopes; resolution: regenerate key in provider console, update environment variable, restart HelixTranslate. Issue 5: "Verification pipeline times out consistently" — diagnosis: check `LLMSVERIFIER_TIMEOUT` value against provider latency, check network path to provider, review per-step timing in pipeline logs; resolution: increase timeout, enable provider retry, or skip non-critical steps. Issue 6: "Score badge shows 0.0 for a known-good model" — diagnosis: check if capability component is 0.0 due to insufficient challenge passes (< 193), check scoring engine error logs; resolution: re-run challenges or reduce minimum challenge threshold in development. Issue 7: "SQLite database locked errors" — diagnosis: check concurrent access from multiple HelixTranslate instances, review `MaxConcurrentTests` setting; resolution: use PostgreSQL backend for shared state, reduce concurrency, or implement connection pooling. Issue 8: "Translation uses unverified model despite gatekeeper enabled" — diagnosis: check if user override was activated, verify gatekeeper is not bypassed in API path; resolution: disable user override in config, add middleware check. Issue 9: "Event bus missing verifier events" — diagnosis: check `EventIntegrationConfig.Enabled`, verify event type subscriptions in `internal/verifier/event_subscriber.go`; resolution: enable event integration, restart subscriber. Issue 10: "High memory usage from score caching" — diagnosis: check cache size in `ScoreCache.entries`, review TTL settings; resolution: reduce `LLMSVERIFIER_CACHE_TTL`, enable cache size limits, or switch to Redis-backed cache.

9.4 Operational Runbook

The operational runbook defines standard operating procedures for the four most common incident types. Each procedure follows detect-diagnose-remediate-verify pattern.

9.4.1 Monitoring Metrics Dashboard

Table 9.4 defines the metrics that operators monitor, their PromQL-style query expressions, alert thresholds, severity levels, and runbook cross-references. In environments without Prometheus, equivalent queries against the SQLite metrics table in `internal/verifier/scoring/history.go` apply.

Table 9.4: Monitoring Metrics Dashboard

Metric Name	Query / Check	Threshold	Severity
<code>verifier_pipeline_failures_total</code>	<code>SELECT COUNT(*) FROM verification_results WHERE passed = false AND</code>	> 5 per hour	P2 (High)

Metric Name	Query / Check	Threshold	Severity
model_score_drop_percent	<pre> created_at > now() - interval '1 hour' SELECT (prev_score - curr_score) / prev_score * 100 FROM score_history WHERE model_id = \$1 ORDER BY timestamp DESC LIMIT 2 </pre>	> 10% within 1 hour	P1 (Critical)
discovery_service_unavailable	<pre> SELECT COUNT(*) FROM discovery_runs WHERE status = 'failed' AND provider = \$1 AND created_at > now() - interval '30 minutes' </pre>	> 3 consecutive failures	P2 (High)
gatekeeper_denial_rate	<pre> SELECT COUNT(*) FROM gatekeeper_decisions WHERE allowed = false AND created_at > now() - interval '1 hour' / total requests </pre>	> 20% of requests	P3 (Medium)
verifier_api_latency_p95	<pre> SELECT percentile_cont(0.95) WITHIN GROUP (ORDER BY duration_ms) FROM api_calls WHERE endpoint LIKE '/api/v1/verify%' AND created_at > now() - interval '5 minutes' </pre>	> 5000ms	P2 (High)
sqlite_db_size_bytes	<pre> stat -c%s /data/ verifier.db </pre>	> 1GB	P3 (Medium)
challenge_pass_rate	<pre> SELECT passed_count / total_count * 100 FROM challenge_runs WHERE model_id = \$1 AND created_at > now() - interval '24 hours' </pre>	< 80% for any verified model	P2 (High)
provider_api_error_rate	<pre> SELECT COUNT(*) FROM provider_calls WHERE status >= 500 AND created_at > now() - interval '5 minutes' / total calls </pre>	> 5%	P3 (Medium)
score_cache_hit_rate	<pre> cache_hits / (cache_hits + cache_misses) </pre>	< 50%	P4 (Low)
rollback_activation_time_ms	Time from LLMSVERIFIER_ENABLED=false to last verified model request	> 30000ms	P1 (Critical)

9.4.2 Key Log Patterns

Operators grep for five log patterns during incident diagnosis. Pattern 1 — successful verification: `msg="verification pipeline passed"` `model_id="<id>"` `steps_passed=8/8` `duration_ms=<n>` appears at INFO level in `internal/verifier/pipeline.go` when all eight steps complete. Pattern 2 — score change: `msg="model score updated"` `model_id="<id>"` `previous=<old>` `current=<new>` `delta=<delta>` appears at INFO level in `internal/verifier/scoring/composite.go` whenever a score recalculation produces a different composite value. Pattern 3 — model retirement: `msg="model retired from registry"` `model_id="<id>"` `reason="score_below_threshold"` `score=<score>` `threshold=<threshold>` appears at WARN level in `internal/verifier/discovery/gatekeeper.go` when a model's score drops below its tier threshold for more than the grace period. Pattern 4 — discovery failure: `msg="discovery failed for provider"` `provider="<name>"` `error="<message>"` `consecutive_failures=<n>` appears at ERROR level in `internal/verifier/discovery/service.go`; three consecutive occurrences trigger the fallback to static registry. Pattern 5 — gatekeeping denial: `msg="gatekeeper denied model access"` `model_id="<id>"` `reason="<verification_failed|score_below_threshold|provider_unhealthy>"` appears at WARN level in `internal/verifier/discovery/gatekeeper.go`.

9.4.3 Runbook: Model Score Dropped Below Threshold

Detection: The `model_score_drop_percent` alert fires when any verified model's composite score drops by more than 10% within one hour, or the `verifier_pipeline_failures_total` alert fires when more than five pipelines fail in one hour. Both alerts route to the on-call engineer via PagerDuty with severity P1.

Diagnosis: The on-call engineer runs three queries in sequence. First, fetch the component-level breakdown: `SELECT component, score FROM score_components WHERE model_id = $1 ORDER BY calculated_at DESC LIMIT 5` to identify which component (speed, cost, efficiency, capability, recency) drove the drop. Second, check the model's recent challenge results: `SELECT challenge_id, passed FROM challenge_results WHERE model_id = $1 AND created_at > now() - interval '24 hours'` to determine if a provider-side degradation affected capability scores. Third, review provider health: `SELECT status, latency_ms FROM provider_health WHERE provider = $1 ORDER BY checked_at DESC LIMIT 10` to confirm the provider API is responsive.

Remediation: If the score drop is due to stale data (score older than cache TTL), trigger a forced recalculation via `POST /api/v1/score/calculate` with the model ID. If the drop reflects genuine provider degradation and the model is non-critical, allow the gatekeeper to retire it automatically by waiting for the grace period (default 15 minutes). If the model is critical for production workloads, temporarily lower the minimum score threshold via `LLMSVERIFIER_MIN_SCORE` environment variable (requires service restart) while the provider resolves the issue. Document all actions in the incident log.

9.4.4 Runbook: Discovery Service Failure

Detection: The `discovery_service_unavailable` alert fires when a provider's discovery endpoint fails three or more consecutive times within 30 minutes. The alert includes the provider name and last error message.

Diagnosis: Check provider API key validity by running a direct curl to the provider's model list endpoint with the stored key. If the key is valid, check the provider's public status page. Review `internal/verifier/discovery/service.go` logs for the specific HTTP status code and response body from the failed discovery call.

Remediation — Automatic Fallback: The discovery service automatically falls back to the static registry for the affected provider. Models from the last successful discovery remain available but are marked with `source_tier = 'static'`. No operator action is required for fallback activation.

Remediation — Manual Model Registration: If the provider failure is prolonged and critical models are needed, register models manually via `POST /api/v1/models/register` with a JSON payload containing `model_id`, `provider`, `name`, `context_window`, and `capabilities`. Manually registered models receive a provisional score of 5.0 and are tagged with `registration_type = 'manual'`.

9.4.5 Runbook: New Provider Onboarding

Adding provider 26+ to the verified pipeline follows a 10-step procedure. Step 1: Obtain API credentials and add the `{PROVIDER}_API_KEY` environment variable to the deployment configuration. Step 2: Add provider metadata to the static registry in `internal/verifier/discovery/registry.go` including known model IDs, context windows, and endpoint URLs. Step 3: Implement the provider adapter in `internal/providers/llmsverifier/` following the OpenAI-compatible pattern if applicable, or create a native implementation. Step 4: Add the provider to the discovery service configuration in `configs/verifier.yaml` under `providers_to_discover`. Step 5: Deploy to the staging environment and trigger manual discovery via `POST /api/v1/models/discover`. Step 6: Run the 8-step verification pipeline against each discovered model. Step 7: Verify that all models score above the minimum threshold for their tier; if not, investigate failing components. Step 8: Add provider-specific challenges to the Challenges submodule if the provider introduces novel capabilities. Step 9: Update user-facing documentation in `docs/llmsverifier-integration.md` with the new provider's name, supported models, and pricing tier. Step 10: Enable the provider in production via feature flag and monitor for 24 hours before declaring onboarding complete.

9.5 Deployment Configuration

The LLMsVerifier service deploys as a sidecar to the main HelixTranslate application, communicating over HTTP on port 8080 with an SQLite database persisted to a Docker volume.

9.5.1 Docker Compose Service Definition

The `docker-compose.yml` file at the project root receives a new `llms-verifier` service. This service uses the `helix-translate/llms-verifier:latest` image built from the `./LLMsVerifier/` directory, exposes port 8080, mounts a named volume `verifier-data` to `/data` for SQLite persistence, connects to the existing `helix-network` bridge network, and defines a health check endpoint. Resource limits of 2 CPU cores and 2GB memory apply for production; these are overridden per environment in Section 9.5.4.

9.5.2 Docker Compose Verifier Service

Code block 9.2 shows the complete verifier service definition for `docker-compose.yml`.

```
llms-verifier:
  image: helix-translate/llms-verifier:latest
  build:
    context: ./LLMsVerifier
    dockerfile: Dockerfile
  container_name: helix-llms-verifier
  ports:
    - "8080:8080"
  volumes:
    - verifier-data:/data
  networks:
    - helix-network
  environment:
    - DB_PATH=/data/verifier.db
    - API_PORT=8080
    - LOG_LEVEL=info
    - LLMsverifier_API_KEY=${LLMSVERIFIER_API_KEY}
    - OPENAI_API_KEY=${OPENAI_API_KEY}
    - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
    - GROQ_API_KEY=${GROQ_API_KEY}
    - MISTRAL_API_KEY=${MISTRAL_API_KEY}
    - COHERE_API_KEY=${COHERE_API_KEY}
    - XAI_API_KEY=${XAI_API_KEY}
    - TOGETHER_API_KEY=${TOGETHER_API_KEY}
    - SAMBANOVA_API_KEY=${SAMBANOVA_API_KEY}
    - NOVITA_API_KEY=${NOVITA_API_KEY}
    - QWEN_API_KEY=${QWEN_API_KEY}
    - REPLICATE_API_TOKEN=${REPLICATE_API_TOKEN}
    - HYPERBOLIC_API_KEY=${HYPERBOLIC_API_KEY}
    - CEREBRAS_API_KEY=${CEREBRAS_API_KEY}
    - SILICONFLOW_API_KEY=${SILICONFLOW_API_KEY}
    - FIREWORKS_API_KEY=${FIREWORKS_API_KEY}
    - PERPLEXITY_API_KEY=${PERPLEXITY_API_KEY}
    - AI21_API_KEY=${AI21_API_KEY}
    - ZHIPU_API_KEY=${ZHIPU_API_KEY}
  healthcheck:
```

```

    test: ["CMD", "wget", "-q", "--spider", "http://
      localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 15s
  deploy:
    resources:
      limits:
        cpus: '2'
        memory: 2G
      reservations:
        cpus: '1'
        memory: 512M
    restart: unless-stopped
    depends_on:
      - api-server

```

The named volume `verifier-data` must be added to the `volumes` section at the bottom of `docker-compose.yml`:

```

volumes:
  verifier-data:
    driver: local

```

9.5.3 Kubernetes Manifests

Three Kubernetes manifests deploy the verifier service in production. The `verifier-deployment.yml` defines a Deployment with two replicas for high availability, liveness and readiness probes against `/health`, pod anti-affinity to spread replicas across nodes, and resource requests/limits. The `verifier-service.yml` exposes the deployment as a ClusterIP service on port 8080. The `verifier-configmap.yml` holds non-sensitive configuration: `API_PORT`, `LOG_LEVEL`, `DB_PATH`, scoring weights, discovery intervals, and verification timeouts. API keys are injected via a Kubernetes Secret named `llmsverifier-api-keys` mounted as environment variables from the deployment's `envFrom` directive.

Code block 9.3 shows the complete `verifier-deployment.yml` manifest.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: llms-verifier
  namespace: helix-translate
  labels:
    app: llms-verifier
    component: verification
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llms-verifier

```

```
template:
  metadata:
    labels:
      app: llms-verifier
      component: verification
  spec:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - llms-verifier
              topologyKey: kubernetes.io/hostname
    containers:
      - name: verifier
        image: helix-translate/llms-verifier:latest
        ports:
          - containerPort: 8080
            name: http
        envFrom:
          - configMapRef:
              name: verifier-config
          - secretRef:
              name: llmsverifier-api-keys
        volumeMounts:
          - name: verifier-data
            mountPath: /data
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 15
          periodSeconds: 30
          timeoutSeconds: 10
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 2
        resources:
          requests:
            cpu: "1"
            memory: "1Gi"
          limits:
```

```

      cpu: "2"
      memory: "2Gi"
    volumes:
      - name: verifier-data
        persistentVolumeClaim:
          claimName: verifier-data-pvc

```

9.5.4 Resource Requirements Per Environment

Table 9.5 specifies CPU, memory, storage, replica count, and high-availability configuration for each deployment environment.

Table 9.5: Resource Requirements Per Environment

Environment	CPU Request	CPU Limit	Memory Request	Memory Limit	Storage	Replicas	HA Config	Notes
Development	0.5	1.0	512MB	1GB	SQLite local file, 100MB	1	None	Shares Docker network with a server reload enable
Staging	1.0	2.0	1GB	2GB	SQLite on PVC, 500MB	1	Single pod, daily backup	Mirror production config; reduce scale; verification pipeline enable
Production	2.0	4.0	2GB	4GB	SQLite on PVC (or PostgreSQL), 2GB	2	Pod anti-affinity across zones	Persistent volume with snapshot backup; read replica score queries

9.6 Migration and Rollback Plan

The migration from the legacy provider factory to the LLMsVerifier-backed pipeline uses a feature flag for gradual, reversible rollout.

9.6.1 Migration Script

The `scripts/migrate-to-verifier.sh` script automates the rollout. It checks prerequisites (database connectivity, feature flag availability, verifier service health), runs the database migration to create `verified_models` and `score_history` tables, copies existing model configurations from `internal/config/` into the verifier registry with initial scores computed from default weights, and toggles the feature flag. Code block 9.4 shows the script.

```
#!/usr/bin/env bash
# scripts/migrate-to-verifier.sh
# Gradual migration from legacy provider factory to LLMsVerifier-
  backed pipeline.

set -euo pipefail

PHASE=${1:-"canary"} # canary | early-access | majority | full
DB_PATH=${DB_PATH:-"./data/verifier.db"}
API_URL=${LLMSVERIFIER_API_URL:-"http://localhost:8080"}
FEATURE_FLAG_KEY="use_verified_pipeline"

echo "=== LLMsVerifier Migration: $PHASE ==="

# Step 1: Verify prerequisites
echo "[1/6] Checking prerequisites..."
if ! curl -sf "${API_URL}/health" > /dev/null 2>&1; then
    echo "ERROR: LLMsVerifier health check failed at $API_URL"
    exit 1
fi

# Step 2: Run database migration
echo "[2/6] Running database migration..."
sqlite3 "$DB_PATH" << 'EOF'
CREATE TABLE IF NOT EXISTS verified_models (
    model_id TEXT PRIMARY KEY,
    provider TEXT NOT NULL,
    name TEXT NOT NULL,
    verified_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    verification_status TEXT DEFAULT 'pending',
    composite_score REAL DEFAULT 0.0,
    component_speed REAL DEFAULT 0.0,
    component_cost REAL DEFAULT 0.0,
    component_efficiency REAL DEFAULT 0.0,
    component_capability REAL DEFAULT 0.0,
    component_recency REAL DEFAULT 0.0,
    tier TEXT DEFAULT 'standard',
    capabilities TEXT,
    source_tier TEXT DEFAULT 'static'
);

CREATE TABLE IF NOT EXISTS score_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_id TEXT NOT NULL,
```

```

        composite_score REAL NOT NULL,
        calculated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (model_id) REFERENCES verified_models(model_id)
    );
CREATE INDEX IF NOT EXISTS idx_score_history_model_time
    ON score_history(model_id, calculated_at DESC);

CREATE TABLE IF NOT EXISTS discovery_runs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    provider TEXT NOT NULL,
    status TEXT NOT NULL,
    models_found INTEGER DEFAULT 0,
    error TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
EOF

```

Step 3: Import existing model configs

```

echo "[3/6] Importing existing model configurations..."
go run scripts/import_model_configs.go --db="$DB_PATH" --
    config="config.json"

```

Step 4: Compute initial scores

```

echo "[4/6] Computing initial scores..."
curl -sf -X POST "${API_URL}/api/v1/score/calculate-all" \
    -H "Authorization: Bearer ${LLMSVERIFIER_API_KEY}" || {
    echo "WARNING: Bulk score calculation failed; scores will be
        computed on demand"
}

```

Step 5: Set feature flag percentage

```

echo "[5/6] Setting feature flag for phase: $PHASE..."
case "$PHASE" in
    canary)
        PERCENTAGE=5
        ;;
    early-access)
        PERCENTAGE=25
        ;;
    majority)
        PERCENTAGE=50
        ;;
    full)
        PERCENTAGE=100
        ;;
    *)
        echo "ERROR: Unknown phase: $PHASE"
        exit 1
        ;;
esac

```

Step 6: Verify rollout

```

echo "[6/6] Verifying rollout at ${PERCENTAGE}%..."

```

```

sleep 5
curl -sf "${API_URL}/api/v1/status?feature_flag=${FEATURE_FLAG_KEY}" | \
jq -e ".rollout_percentage == ${PERCENTAGE}" || {
  echo "ERROR: Feature flag verification failed"
  exit 1
}

echo "=== Migration to $PHASE (${PERCENTAGE}%) completed successfully ==="

```

9.6.2 Rollout Phases

The rollout proceeds through four phases with explicit entry and exit criteria. Phase 1 — Canary (5% traffic, 1 day): deploy to 5% of production traffic. Exit criteria: error rate remains below 0.1% and no P1/P2 alerts fire for 24 hours. Phase 2 — Early Access (25% traffic, 3 days): expand to 25% of traffic. Exit criteria: user satisfaction score exceeds 4.0 out of 5.0 from feedback surveys, and average translation latency does not regress by more than 5% compared to the legacy pipeline. Phase 3 — Majority (50% traffic, 1 week): expand to 50% of traffic. Exit criteria: all performance SLAs met (p95 latency < 5s, cache hit rate > 80%), and the verification pipeline completes successfully for at least 95% of models. Phase 4 — Full (100% traffic, 2 weeks): complete cutover. Exit criteria: all quality gates pass (100% test coverage confirmed, all 193+ challenges passing, all 14 documentation files complete), and no critical incidents for 14 consecutive days.

9.6.3 Rollback Procedure

Rollback is instantaneous and requires zero downtime. The feature flag `LLMSVERIFIER_ENABLED` controls whether `pkg/translator/llm/llm.go` uses the verified pipeline or the legacy provider factory. Setting `LLMSVERIFIER_ENABLED=false` causes the next provider factory invocation to bypass `internal/verifier/discovery/gatekeeper.go` and construct the provider directly from `internal/config/config.go` as in the pre-integration codebase. Code block 9.5 shows the rollback toggle.

```

#!/usr/bin/env bash
# scripts/rollback-to-legacy.sh
# Instant rollback from LLMSVerifier to legacy provider factory.

set -euo pipefail

echo "=== Initiating emergency rollback ==="

# Method 1: Environment variable (takes effect on next request, no
# restart)
export LLMSVERIFIER_ENABLED=false
curl -X POST "http://localhost:8080/admin/feature-flags" \
  -H "Authorization: Bearer ${ADMIN_API_KEY}" \
  -d '{"LLMSVERIFIER_ENABLED": false}'

# Method 2: Config file update (persists across restarts)
jq '.llmsverifier.enabled = false' config.json > config.json.tmp

```

```

mv config.json.tmp config.json

# Verify rollback
echo "Verifying rollback..."
sleep 2
LEGACY_CALLS=$(curl -sf "http://localhost:8080/metrics" | \
  grep 'provider_factory_legacy_calls_total' | tail -1 | awk
    '{print $2}')
if [[ "$LEGACY_CALLS" -gt 0 ]]; then
  echo "Rollback verified: legacy provider factory is active"
else
  echo
    "WARNING: Rollback verification inconclusive; check
    metrics manually"
fi

echo "=== Rollback complete ==="

```

9.6.4 Data Migration

Existing model configurations in `internal/config/config.go` migrate to the `verified_models` SQLite table. The migration preserves provider name, model ID, API key reference (not the key itself), base URL, and assigned tier. Each migrated model receives an initial composite score of 5.0 (the "fair" baseline) and a `source_tier` value of `migrated`. Scores are recomputed through the full 5-component pipeline within 24 hours of migration. The migration is idempotent: running `scripts/migrate-to-verifier.sh` multiple times does not duplicate entries because the `model_id` column is the primary key.

9.7 Final Checklist and Definition of Done

9.7.1 Master Checklist

The master checklist contains 50 items across all eight phases. Each item carries a status column (pending / in-progress / complete), owner assignment (engineer name or team), and a sign-off signature with date. Phase 1 (Foundation) contains 8 items: module integration in `go.mod`, directory structure creation, `LLMsVerifierConfig` struct definition, environment variable loading, interface alignment in `internal/verifier/bridge.go`, constitution file creation, `CLAUDE.md` update, and `AGENTS.md` update. Phase 2 (Verification) contains 8 items: `NewVerifierClient()` implementation, health check endpoint, 8-step pipeline definition, existence verification, responsive/feature detection, code visibility and coding challenges, performance benchmarking, and cost analysis integration. Phase 3 (Scoring) contains 6 items: scoring engine initialization, five-component score calculation, composite score aggregation, score adapter service, score persistence and history, and threshold management. Phase 4 (Discovery) contains 6 items: 3-tier discovery service, provider API enumeration, model registry CRUD, gatekeeper implementation, background sync runner, and model catalog API endpoints. Phase 5 (Runtime) contains 6 items: verified provider factory, runtime model selection engine, fallback chain, event bus integration, API key validation, and provider

expansion to 30+. Phase 6 (UX) contains 4 items: model selection interface, score badge rendering, filtering system, and unverified model warning flow. Phase 7 (Testing) contains 7 items: unit tests for verifier client, pipeline tests, scoring engine tests, score adapter tests, discovery and registry tests, integration tests with live verifier, and anti-bluff challenge suite (4 challenges, 193+ total). Phase 8 (Documentation) contains 5 items: all constitution files signed, all CLAUDE.md and AGENTS.md updates merged, user-facing documentation published, operational runbook reviewed by on-call team, and deployment manifests validated in staging.

9.7.2 Quality Gates

Three quality gates block final sign-off. Gate 1 — Test Coverage: `go test -cover ./internal/verifier/...` must report 100% statement coverage. The coverage report is generated with `go test -coverprofile=coverage.out ./internal/verifier/... && go tool cover -html=coverage.out -o coverage.html` and reviewed by the test lead. Gate 2 — Challenge Pass: all 193+ challenges must pass when executed against a live LLMsVerifier instance with real provider API keys. The challenge run is initiated with `go test -tags=challenge ./internal/verifier/challenges/...` and the output is archived. Gate 3 — Documentation Completeness: all 14 documentation files (3 CONSTITUTION.md, 3 CLAUDE.md, 3 AGENTS.md, 2 README.md, docs/llmsverifier-integration.md, docs/troubleshooting.md, internal/verifier/README.md) must pass the validation script from Section 9.2.4.

9.7.3 Anti-Bluff Verification

The anti-bluff verification is a manual procedure performed once after all automated gates pass. A senior engineer runs the full integration test suite against a live LLMsVerifier instance configured with real (not mocked) provider API keys for at least three distinct providers. The test sequence follows the complete user journey: trigger discovery, verify a model, compute its score, select it for translation, execute a translation, and confirm the model appears in the score history. The engineer pastes terminal output into the sign-off document. If any step produces a result that contradicts the automated test suite, the discrepancy is investigated as a potential test-bluff issue.

9.7.4 Performance Verification

Performance verification confirms all SLA targets under production-like load. The load test generates 1,000 concurrent translation requests using the verified model selection pipeline. Targets: p50 latency below 500ms, p95 latency below 2,000ms, p99 latency below 5,000ms, score cache hit rate above 80%, zero goroutine leaks after 30 minutes of sustained load, and memory growth below 10% over the same 30-minute window. The load test is executed with `go test -tags=perf -run TestVerifierLoad ./test/performance/...` against the staging environment. Results are captured with `go test -memprofile=mem.prof -cpuprofile=cpu.prof` and reviewed for hot paths before sign-off.